Aivan HRMS Phase 1 and Phase 2 Review Before Live Publish

Date: 2026-07-13

Branch reviewed: ‘Phase2-kaushal‘

Reviewed by: Vivek Mehta

Purpose: This report is prepared to share with the frontend and backend developer before publishing the application live. It explains what was checked, what is already done well, what was fixed during review, and what still needs correction before the final production deployment.

1. Review Summary

The ‘Phase2-kaushal‘ branch was reviewed as a production candidate for Phase 1 and Phase 2.

First, the branch was updated from GitHub and checked against ‘Phase1-kaushal‘. The result confirmed that Phase 1 code is already included inside ‘Phase2-kaushal‘, so no additional Phase 1 merge was required.

The codebase was then checked against our Phase 1 and Phase 2 implementation documents. Both frontend and backend now contain the main expected modules for login, admin, HR, employee, attendance, time off, regularization, master data, reports, approval flow, notifications, and login activity.

Overall, a lot of good work has been completed across both frontend and backend. The current branch is ready for staging deployment and very close to live deployment. Before public live publish, a few final server-side and release-hardening fixes must be completed.

2. Developer Appreciation

Kaushal has done strong work across both frontend and backend.

This is especially worth appreciating because the project is not a small UI-only task. It includes role-based login, dashboards, employee management, attendance workflows, time off workflows, regularization, reports, backend APIs, database models, migrations, services, and security-related flows.

For a developer with limited backend experience, the amount of implementation completed here is genuinely good progress. The frontend screens are also visually aligned and the UI flow is in good shape, so no UI redesign was needed during this review.

The remaining work is not a rejection of the work already done. It is the normal final hardening stage before publishing a real HRMS system live.

3. What Was Tested

The following checks were performed:

1. Git branch status was checked.
2. Latest remote code was pulled from ‘origin/Phase2-kaushal‘.
3. ‘Phase1-kaushal‘ inclusion was verified inside ‘Phase2-kaushal‘.
4. Phase 1 frontend/backend scope was compared with current code.
5. Phase 2 frontend/backend scope was compared with current code.
6. Frontend dependencies were installed.
7. Frontend production build was tested.
8. Frontend automated tests were run.
9. Backend Python compile check was run.
10. Backend test environment was created.
11. Backend automated tests were run.
12. Backend production-style import/startup check was run.
13. Alembic migration head was checked.

14. Frontend dependency security audit was run.
15. Hardcoded local API/debug patterns were scanned.
16. Remaining release blockers were documented.

4. Test Results

Frontend

1. ‘npm run build‘
   Result: Passed

2. ‘npm test -- --watch=false‘
   Result: Passed

3. Frontend test count
   Result: 16 test files passed, 17 tests passed

4. ‘npm audit --audit-level=high‘
   Result: Passed

5. Remaining npm audit issue
   Result: 1 low severity development-server-related ‘esbuild‘ advisory remains

Backend

1. Python compile check
   Result: Passed

2. Backend pytest
   Result: Passed

3. Backend test count
   Result: 40 tests passed

4. Backend production-style import with production flags
   Result: Passed

5. Alembic head check
   Result: Passed

6. Alembic current head
   Result: ‘9c76cbfeaf96‘

5. What Was Done Perfectly Or Very Well

1. Phase 1 role flow is present.
   Admin, HR, and employee role flows are implemented in the same portal.

2. Phase 1 code is already included in Phase 2.
   ‘Phase1-kaushal‘ is already part of ‘Phase2-kaushal‘, so the branch is not missing the previous phase foundation.

3. Frontend UI flow is in good shape.
   The UI was reviewed from an implementation perspective, and no visual redesign was required.

4. Frontend production build passes.
   This means Angular can generate a deployable production bundle.

5. Frontend automated tests pass.
   This gives confidence that the existing frontend components can compile and run in the test environment.

6. Backend tests pass.
   40 backend tests passing is a good sign for service logic and API expectations.

7. Backend module structure is aligned with Phase 2.
   Time off, regularization, approval, reports, master data, notifications, login activity, attendance, employees, and HR modules are present.

8. Backend production config is safer now.
   Backend fails if ‘DATABASE_URL‘ or ‘JWT_SECRET_KEY‘ is missing. This is good because production should not start with unsafe defaults.

9. High severity frontend dependency vulnerabilities were reduced.
   Earlier audit showed multiple high severity issues. After dependency hardening, high severity audit check passes.

10. API-driven structure is much stronger than Phase 1 mock mode.
   The code has moved toward real backend/API-driven behavior, which matches our Phase 2 goal.

6. Fixes Already Done During Review

1. Added missing backend test dependency.
   ‘httpx2==2.5.0‘ was added to ‘backend/requirements.txt‘.

   Why this was required:
   Backend tests use FastAPI TestClient. Without ‘httpx2‘, tests failed during collection before actual test execution.

2. Updated Angular dependencies to safe patch versions.
   Angular packages were updated within Angular 21 patch versions.

   Why this was required:
   ‘npm audit‘ showed high severity Angular/build dependency advisories. This was a release security concern.

3. Regenerated frontend package lock.
   ‘frontend/package-lock.json‘ was updated after dependency hardening.

   Why this was required:
   The lockfile must match the dependency versions that were actually tested and should be deployed.

4. Re-tested after dependency fixes.
   Frontend build, frontend tests, backend tests, and high-severity audit were run again after fixes.

   Why this was required:
   Dependency changes can break builds, so the code had to be verified again after updating packages.

7. What Still Needs To Be Fixed Before Live Publish

1. PostgreSQL migration must be tested on the real server database

What needs to be fixed:

1. Create or connect the real PostgreSQL production/staging database.
2. Set the correct ‘DATABASE_URL‘.
3. Run ‘alembic upgrade head‘ on the real PostgreSQL DB.
4. Confirm ‘alembic current‘ shows ‘9c76cbfeaf96‘.

Why this is required:

The local SQLite migration smoke check failed because SQLite does not support one constraint

alteration used in migration. Production is expected to use PostgreSQL, so the final migration test must be done on PostgreSQL.

Risk if not fixed:

The backend may start locally but fail during server deployment when database schema is not created or upgraded correctly.

Priority:

P1 - Must fix before live publish.

2. Production environment variables must be configured correctly

What needs to be fixed:

1. Create backend ‘.env‘ on the server.
2. Add production ‘DATABASE_URL‘.
3. Add strong ‘JWT_SECRET_KEY‘.
4. Set ‘APP_ENV=production‘.
5. Set ‘AUTO_CREATE_TABLES=false‘.
6. Set ‘AUTO_SEED_ROLES=false‘.
7. Set ‘AUTO_SEED_DEMO_DATA=false‘.
8. Set ‘EXPOSE_RESET_LINK_IN_RESPONSE=false‘.
9. Set correct ‘BACKEND_CORS_ORIGINS‘.
10. Set correct ‘FRONTEND_URL‘.

Why this is required:

Production must not use local defaults, demo credentials, or unsafe reset-link behavior.

Risk if not fixed:

Login, CORS, password reset, database connection, or API startup can fail on server.

Priority:

P1 - Must fix before live publish.

3. First production admin account must be handled safely

What needs to be fixed:

1. Decide how first admin will be created on production.
2. Do not leave demo admin credentials active on live server.
3. If temporary seeding is used, change or disable demo credentials before public access.
4. Confirm real admin login before handing over to HR users.

Why this is required:

Without a real admin account, the portal cannot be managed. With demo credentials active, the system becomes unsafe.

Risk if not fixed:

Either the team cannot log in as admin, or public users may access the system using known demo credentials.

Priority:

P1 - Must fix before live publish.

4. IIS reverse proxy must be configured for API and WebSocket

What needs to be fixed:

1. Serve Angular production build from IIS.
2. Proxy ‘/api/*‘ requests to FastAPI backend.
3. Proxy ‘/ws/*‘ WebSocket requests to FastAPI backend.
4. Enable WebSocket support in IIS.
5. Test login, dashboard API calls, notifications, and WebSocket connection from browser.

Why this is required:

Frontend production config uses relative ‘/api/v1‘. This expects frontend and backend to work through the same server/domain.

Risk if not fixed:

Frontend may load, but login/API calls or real-time notifications may fail.

Priority:

P1 - Must fix before live publish.

5. Do final server smoke test after deployment

What needs to be fixed:

1. Open login page on server URL.
2. Login as admin.
3. Open admin dashboard.
4. Create or verify HR user.
5. Login as HR.
6. Create or verify employee.
7. Login as employee.
8. Punch in and punch out.
9. Check attendance history.
10. Check time off and regularization pages.
11. Check reports page.
12. Check browser console for API/WebSocket errors.

Why this is required:

Local build/test passing is not the same as server deployment passing. Server config, IIS proxy, database, and environment variables must be validated together.

Risk if not fixed:

The app may pass locally but fail after being published.

Priority:

P1 - Must fix before live publish.

6. Remove or gate debug console logs

What needs to be fixed:

1. Remove routine ‘console.log‘ statements from employee services.
2. Remove routine modal debug logs.
3. Remove dashboard profile debug logs.
4. Keep only meaningful ‘console.error‘ logs or environment-gated debug logs.

Why this is required:

Production browser console should not expose routine internal flow details.

Risk if not fixed:

The app still works, but it looks less polished and may expose unnecessary internal behavior during support/debugging.

Priority:

P2 - Should fix before final polished production release.

7. Clean backend deprecation warnings

What needs to be fixed:

1. Move Pydantic class-based ‘Config‘ to ‘ConfigDict‘.
2. Replace deprecated ‘json_encoders‘ patterns where needed.
3. Replace FastAPI ‘@app.on_event‘ startup/shutdown with lifespan handlers.

Why this is required:

Tests pass today, but these warnings show future compatibility work is needed.

Risk if not fixed:

Future dependency upgrades may become harder and may break behavior later.

Priority:

P2 - Should fix after live-critical deployment tasks.

8. Track remaining low severity frontend audit issue

What needs to be fixed:

1. Track the remaining ‘esbuild‘ low severity advisory.
2. Resolve it when Angular build chain publishes a compatible patch.
3. Do not run ‘ng serve‘ as production server.

Why this is required:

The remaining advisory is related to development server behavior, not the deployed static production bundle, but it should still be tracked.

Risk if not fixed:

Low risk for production static deployment, higher concern only if development server is exposed publicly.

Priority:

P3 - Track and fix when compatible update is available.

8. Final Developer Notes

Kaushal, frontend and backend work is looking good overall. The main modules expected for Phase 1 and Phase 2 are present, and the code passes build/test verification after dependency hardening.

The next corrections are mainly final production-readiness items:

1. server database migration confirmation
2. production environment setup

3. first admin handling
4. IIS API/WebSocket proxy
5. final server smoke testing
6. debug log cleanup
7. deprecation warning cleanup

This is a strong implementation stage. The remaining work is the final polish and server hardening needed before we put the system live.

9. Current Publish Decision

Staging deployment:

Approved to proceed.

Live production deployment:

Proceed only after P1 fixes are completed and verified on the actual Windows/PostgreSQL server.